МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЗАХІДНОУКРАЇНСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
Факультет комп’ютерних інформаційних технологій
Кафедра комп’ютерних наук

ЗВІТ

про виконання лабораторної роботи №1
з дисципліни «Методи аналізу інтервальних даних»
на тему: «Програмні системи для обчислення виразів та рівнянь, заданих в інтервальному вигляді»
Варіант 1

Виконав:
студент групи ІПЗМПм-11
Боднар А.

Перевірила:
к.т.н., доцент
Крепич С. Я.

Тернопіль – 2026

МЕТА РОБОТИ

Вивчити основні операції інтервальної арифметики над дійсними та комплексними інтервалами; навчитися обчислювати вирази, задані в інтервальному вигляді, з урахуванням пріоритету операцій і дужок; розробити програму, що розбирає такі вирази, будує з їхніх значень комплексні інтервали та виконує над ними арифметичні операції.

ЗАВДАННЯ

На вхід програми подаються два інтервальні вирази. Значення першого виразу X_1 є основою (дійсною частиною) першого комплексного інтервалу, значення другого X_2 — основою другого. Уявна частина кожного комплексного інтервалу утворюється з основи за правилом, заданим у варіанті. Програма має обчислити суму, різницю та добуток двох отриманих комплексних інтервалів.

ВИХІДНІ ДАНІ

Варіант 1 (за порядковим номером у журналі) — таблиця 1.

Таблиця 1 – Вихідні дані варіанту 1

Параметр	Значення
Вираз 1	[1; 4] − [−3; 5] · ([4; 5] + [8; 9]) − [−1; 1] / [4; 5]
Вираз 2	[0; 8] − [−4; 1] · ([3; 5] + ([−1; 2] / [4; 5] − [0; 2]))
Уявна частина	Ділення основ на 3

ТЕОРЕТИЧНІ ВІДОМОСТІ

Інтервалом називається множина дійсних чисел $X = [x_1; x_2] = \{x : x_1 \le x \le x_2\}$. Інтервальні операції визначаються так, щоб результат містив усі можливі значення відповідної дії над числами з інтервалів-операндів:

$$[a_1; a_2] + [b_1; b_2] = [a_1 + b_1; a_2 + b_2],$$

$$[a_1; a_2] - [b_1; b_2] = [a_1 - b_2; a_2 - b_1],$$

$$[a_1; a_2] \cdot [b_1; b_2] = [\min\{a_1b_1, a_1b_2, a_2b_1, a_2b_2\}; \max\{a_1b_1, a_1b_2, a_2b_1, a_2b_2\}],$$

$$[a_1; a_2] / [b_1; b_2] = [\min\{a_1/b_1, a_1/b_2, a_2/b_1, a_2/b_2\}; \max\{\dots\}].$$

Ділення в класичному вигляді визначене, коли дільник не містить нуля. Якщо нуль лежить усередині дільника, програма застосовує те саме правило мінімуму й максимуму чотирьох часток, як у прикладі з лекції, і виводить попередження; якщо ж нулю дорівнює межа дільника, ділення неможливе.

Комплексний інтервал має вигляд $K = X + iY$, де X і Y — дійсні інтервали. Операції над комплексними інтервалами зводяться до операцій над дійсними:

$$K_1 \pm K_2 = (X_1 \pm X_2) + i(Y_1 \pm Y_2),$$

$$K_1 \cdot K_2 = (X_1X_2 - Y_1Y_2) + i(X_1Y_2 + Y_1X_2).$$

Вирази обчислюються з урахуванням звичайного пріоритету: спочатку дужки, потім множення й ділення, потім додавання й віднімання; операції одного рівня виконуються зліва направо.

ХІД РОБОТИ

1. Обчислення першого виразу

Програма виконує операції у порядку, заданому дужками й пріоритетами. Кроки обчислення виразу 1 наведено в таблиці 2.

Наприклад, добуток $[-3; 5] \cdot [12; 14]$ визначається серед чотирьох добутків кінців $\{-36; -42; 60; 70\}$: мінімум і максимум дають $[-42; 70]$.

Таблиця 2 – Покрокове обчислення виразу 1

№	Операція	Результат
1	[4; 5] + [8; 9]	[12; 14]
2	[−3; 5] · [12; 14]	[−42; 70]
3	[1; 4] − [−42; 70]	[−69; 46]
4	[−1; 1] / [4; 5]	[−0.25; 0.25]
5	[−69; 46] − [−0.25; 0.25]	[−69.25; 46.25]

$$X_1 = [-69.25; 46.25].$$

2. Обчислення другого виразу

Кроки обчислення виразу 2 наведено в таблиці 3.

Ділення [−1; 2] / [4; 5] виконується через чотири частки кінців {−0.25; −0.2; 0.5; 0.4}, звідки [−0.25; 0.5].

Таблиця 3 – Покрокове обчислення виразу 2

№	Операція	Результат
1	[−1; 2] / [4; 5]	[−0.25; 0.5]
2	[−0.25; 0.5] − [0; 2]	[−2.25; 0.5]
3	[3; 5] + [−2.25; 0.5]	[0.75; 5.5]
4	[−4; 1] · [0.75; 5.5]	[−22; 5.5]
5	[0; 8] − [−22; 5.5]	[−5.5; 30]

$$X_2 = [-5.5; 30].$$

3. Формування комплексних інтервалів

За правилом варіанту («Ділення основ на 3») уявна частина дорівнює Y = X / 3:

$$Y_1 = [-69.25; 46.25] / 3 = [-23.0834; 15.4167],$$

$$Y_2 = [-5.5; 30] / 3 = [-1.8334; 10].$$

Отже,

$$K_1 = [-69.25; 46.25] + i[-23.0834; 15.4167],$$

$$K_2 = [-5.5; 30] + i[-1.8334; 10].$$

4. Операції над комплексними інтервалами

Сума і різниця обчислюються окремо для дійсних і уявних частин:

$$K_1 + K_2 = ([-69.25; 46.25] + [-5.5; 30]) + i([-23.0834; 15.4167] + [-1.8334; 10]) = [-74.75; 76.25] + i[-24.9167; 25.4167],$$

$$K_1 - K_2 = ([-69.25; 46.25] - [-5.5; 30]) + i([-23.0834; 15.4167] - [-1.8334; 10]) = [-99.25; 51.75] + i[-33.0834; 17.25].$$

Для добутку спочатку обчислено чотири добутки дійсних інтервалів:

$$X_1X_2 = [-2077.5; 1387.5], \quad Y_1Y_2 = [-230.8334; 154.1667],$$

$$X_1Y_2 = [-692.5; 462.5], \quad Y_1X_2 = [-692.5; 462.5],$$

звідки

$$K_1 \cdot K_2 = ([-2077.5; 1387.5] - [-230.8334; 154.1667]) + i([-692.5; 462.5] + [-692.5; 462.5]) = [-2231.6667; 1618.3334] + i[-1385; 925].$$

Зведені результати наведено в таблиці 4.

Таблиця 4 – Результати обчислень

Величина	**Значення**
X_1	[−69.25; 46.25]
X_2	[−5.5; 30]
K_1	[−69.25; 46.25] + i[−23.0834; 15.4167]
K_2	[−5.5; 30] + i[−1.8334; 10]
$K_1 + K_2$	[−74.75; 76.25] + i[−24.9167; 25.4167]
$K_1 - K_2$	[−99.25; 51.75] + i[−33.0834; 17.25]
$K_1 \cdot K_2$	[−2231.6667; 1618.3334] + i[−1385; 925]

5. Програмна реалізація

Програму реалізовано як вебзастосунок мовою TypeScript з бібліотеками React і React-Bootstrap; він працює в браузері й розміщений на GitHub Pages (посилання наведено нижче). Обчислювальна частина не залежить від інтерфейсу:

– interval.ts — клас Interval з операціями додавання, віднімання, множення і ділення (з перевіркою нуля в дільнику);
– complex.ts — клас ComplexInterval і операції над комплексними інтервалами;
– parser.ts — розбір виразу методом рекурсивного спуску з урахуванням дужок, пріоритетів і унарного мінуса; кожна виконана операція записується в покрокову трасу, а помилки введення повідомляються з позицією символу;
– variants.ts і lab1.ts — 30 варіантів завдання, правила формування уявної частини та функція solveLab1;
– format.ts — виведення меж з округленням назовні, щоб показаний інтервал містив точний.

Користувач обирає варіант (за замовчуванням — свій) або вводить власні вирази; результати, покрокова таблиця й графічне зображення інтервалів на

комплексній площині оновлюються одразу. Обчислювальну частину перевірено автоматичними тестами на прикладах з лекцій і на всіх 30 варіантах.

Сторінка роботи на сайті: https://4ndruxa.github.io/university-labs/#/maid/lab1.

Репозиторій з кодом: https://github.com/4ndruxa/university-labs.

6. Результати роботи програми

На рисунку 1 наведено вікно введення вхідних даних.

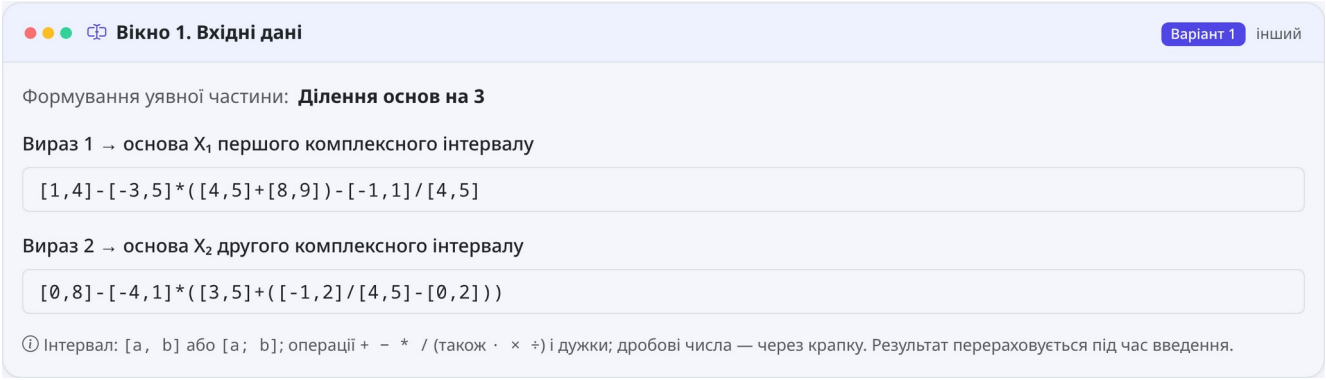


Рисунок 1 – Вікно вхідних даних

На рисунку 2 наведено вікно з результатами обчислень.

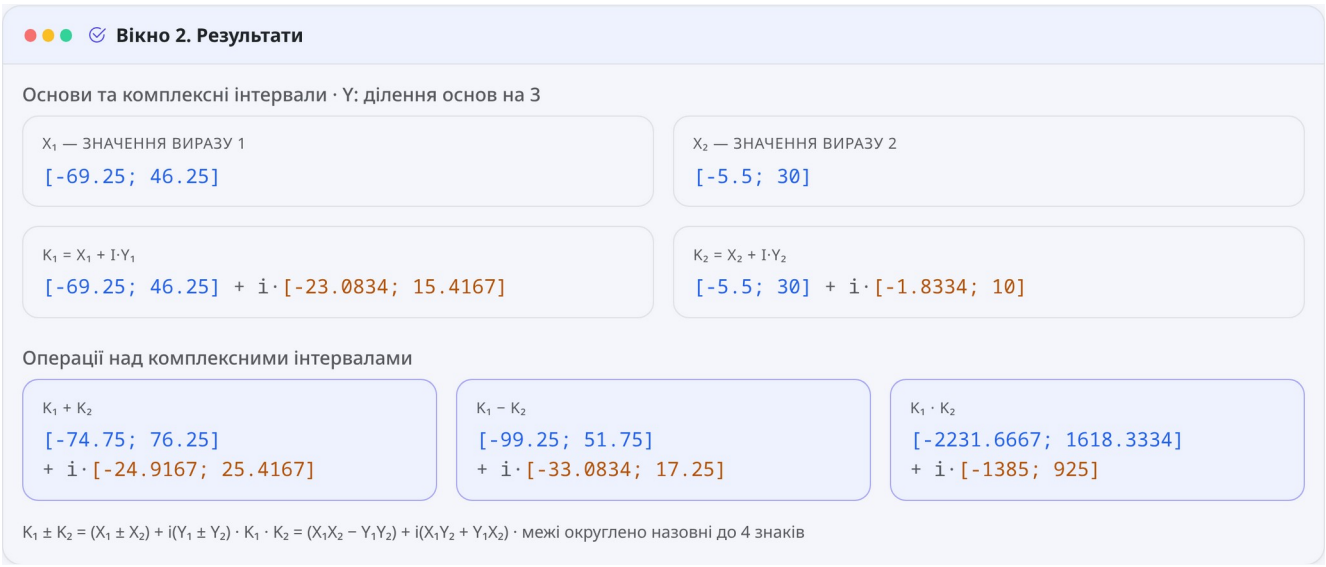


Рисунок 2 – Вікно результатів

На рисунку 3 наведено вікно покрокового обчислення виразів.



Рисунок 3 – Вікно покрокового обчислення

На рисунку 4 наведено графічне зображення інтервалів: комплексні інтервали показано прямокутниками на комплексній площині.

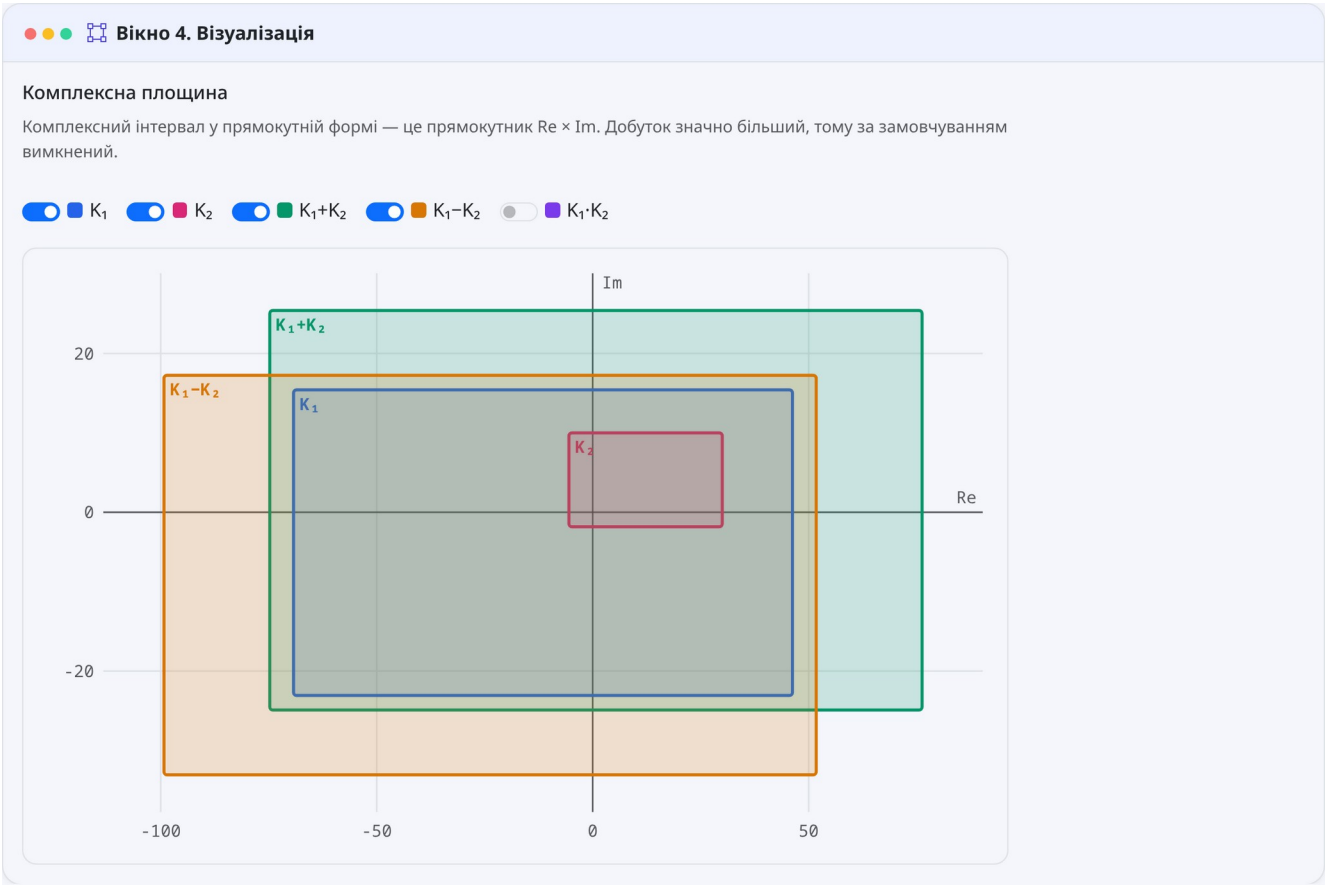


Рисунок 4 – Вікно візуалізації

ВИСНОВКИ

У ході виконання лабораторної роботи опрацьовано правила арифметики дійсних і комплексних інтервалів та розроблено вебзастосунок, який розбирає

інтервальні вирази, показує кожен крок обчислення, формує з результатів комплексні інтервали і виконує над ними додавання, віднімання та множення.

Для варіанту 1 отримано $X_1 = [-69.25; 46.25]$ і $X_2 = [-5.5; 30]$. Ширина результатів (115.5 і 35.5) значно більша за ширину вихідних інтервалів, бо кожна інтервальна операція враховує найгірше поєднання меж. Особливо помітно це в добутку $K_1 \cdot K_2 = [-2231.6667; 1618.3334] + i[-1385; 925]$: різниця і сума добутків, що містять нуль, дають широкі інтервали навколо нуля.

Покрокова траса дозволяє перевірити кожну дію вручну, а результати програми збігаються з ручними обчисленнями, наведеними вище.

ДОДАТОК А. ЛІСТИНГ ПРОГРАМНОГО КОДУ

Нижче наведено обчислювальну частину програми. Повний код застосунку, разом з інтерфейсом і тестами, розміщено в репозиторії https://github.com/4ndruxa/university-labs.

Файл lab1.ts

```
// ЛР1: K = X + i·Y, де X — значення виразу, Y — за правилом варіанту
import { ComplexInterval } from './complex'
import { Interval, type Warn } from './interval'
import { evaluate, type EvalResult } from './parser'
import type { ImagRule } from './variants'

export function imagPart(x: Interval, rule: ImagRule, warn?: Warn): Interval {
  const arg = Interval.from(rule.arg)
  switch (rule.op) {
    case '+': return x.add(arg)
    case '-': return x.sub(arg)
    case '*': return x.mul(arg)
    case '/': return x.div(arg, warn)
  }
}

export interface Lab1Result {
  expr1: EvalResult
  expr2: EvalResult
  k1: ComplexInterval
  k2: ComplexInterval
  sum: ComplexInterval
  diff: ComplexInterval
  prod: ComplexInterval
  warnings: string[]
}

export function solveLab1(e1: string, e2: string, rule: ImagRule): Lab1Result {
  const expr1 = evaluate(e1)
  const expr2 = evaluate(e2)
  const warnings = [
    ...expr1.warnings.map((w) => `Вираз 1: ${w}`),
    ...expr2.warnings.map((w) => `Вираз 2: ${w}`),
  ]
  const warn: Warn = (w) => warnings.push(`Уявна частина: ${w}`)
  const k1 = new ComplexInterval(expr1.value, imagPart(expr1.value, rule, warn))
  const k2 = new ComplexInterval(expr2.value, imagPart(expr2.value, rule, warn))
  return { expr1, expr2, k1, k2, sum: k1.add(k2), diff: k1.sub(k2), prod:
    k1.mul(k2), warnings }
}
```

Файл parser.ts

```
// Розбір і обчислення інтервальних виразів.
//
//   expr     := term (('+' | '−') term)*
//   term     := unary (('·' | '/') unary)*
//   unary    := ('−' | '+') unary | primary
//   primary  := interval | number | '(' expr ')'
```

```
//   interval := '[' signed (',' | ';') signed ']'
//
// Кожна виконана операція записується в trace.
import { Interval } from './interval'

export type BinaryOp = '+' | '-' | '*' | '/'

export type Step =
  | { kind: 'binary'; op: BinaryOp; a: Interval; b: Interval; result: Interval;
    candidates?: number[] }
  | { kind: 'neg'; a: Interval; result: Interval }

export interface EvalResult {
  value: Interval
  trace: Step[]
  warnings: string[]
}

/** pos — індекс символу (з 0) */
export class ParseError extends Error {
  readonly pos: number
  constructor(message: string, pos: number) {
    super(message)
    this.name = 'ParseError'
    this.pos = pos
  }
}

type Token =
  | { kind: 'num'; value: number; pos: number }
  | { kind: 'sym'; value: string; pos: number }
  | { kind: 'end'; pos: number }

const SYMBOL_ALIASES: Record<string, string> = { '−': '-', '–': '-', '·': '*',
    '×': '*', '÷': '/', ':': '/' }
const SYMBOLS = '+-*/()[],;'

function describe(t: Token): string {
  if (t.kind === 'end') return 'кінець виразу'
  if (t.kind === 'num') return `число ${t.value}`
  return `«${t.value}»`
}

export function tokenize(src: string): Token[] {
  const tokens: Token[] = []
  let i = 0
  while (i < src.length) {
    const c = SYMBOL_ALIASES[src[i]] ?? src[i]
    if (/\s/.test(c)) { i++; continue }
    if (SYMBOLS.includes(c)) { tokens.push({ kind: 'sym', value: c, pos: i }); i+
    +; continue }
    const m = /^(?:\d+(?:\.\d*)?|\.\d+)/.exec(src.slice(i))
    if (m) {
      const value = Number(m[0])
      if (!Number.isFinite(value)) throw new ParseError('Число занадто велике', i)
      const next = i + m[0].length
      if (src[next] === 'e' || src[next] === 'E') throw new
    ParseError('Експоненційний запис не підтримується — пишіть, напр., 1000',
    next)
      tokens.push({ kind: 'num', value, pos: i })
```

```
      i = next
      continue
    }
    throw new ParseError(`Невідомий символ «${src[i]}» (дробові числа пишуться
    через крапку: 0.5)`, i)
  }
  tokens.push({ kind: 'end', pos: src.length })
  return tokens
}

export function evaluate(src: string): EvalResult {
  const tokens = tokenize(src)
  const trace: Step[] = []
  const warnings: string[] = []
  let p = 0

  const peek = (): Token => tokens[p]
  const isSym = (...values: string[]): boolean => {
    const t = peek()
    return t.kind === 'sym' && values.includes(t.value)
  }
  const fail = (expected: string): never => {
    const t = peek()
    throw new ParseError(`Очікувалося ${expected}, а знайдено ${describe(t)}`,
    t.pos)
  }
  const expectSym = (value: string, expected = `«${value}»`): void => {
    if (!isSym(value)) fail(expected)
    p++
  }

  function signedNumber(): number {
    let sign = 1
    if (isSym('-')) { p++; sign = -1 } else if (isSym('+')) p++
    const t = peek()
    if (t.kind !== 'num') return fail('число')
    p++
    return sign * t.value
  }

  // pos — початок правого операнда, щоб помилка ділення вказувала на дільник
  function apply(op: BinaryOp, a: Interval, b: Interval, pos: number): Interval {
    let result: Interval
    let candidates: number[] | undefined
    try {
      switch (op) {
        case '+': result = a.add(b); break
        case '-': result = a.sub(b); break
        case '*': candidates = a.products(b); result = a.mul(b); break
        case '/': result = a.div(b, (w) => warnings.push(w)); candidates =
    a.quotients(b); break
      }
    } catch (e) {
      throw new ParseError((e as Error).message, pos)
    }
    trace.push({ kind: 'binary', op, a, b, result, candidates })
    return result
  }

  function primary(): Interval {
    const t = peek()
    if (isSym('[')) {
```

```
    p++
    const lo = signedNumber()
    if (isSym(',', ';')) p++; else fail('«,» або «;» між межами інтервалу')
    const hi = signedNumber()
    if (isSym(';', ',')) fail('«]» (дробові числа пишуться через крапку: [0.5;
  1])')
    expectSym(']')
    if (lo > hi) throw new ParseError(`Некоректний інтервал [${lo}; ${hi}]: ліва
  межа більша за праву`, t.pos)
    return new Interval(lo, hi)
  }
  if (isSym('(')) {
    p++
    const v = expr()
    expectSym(')')
    return v
  }
  if (t.kind === 'num') { p++; return new Interval(t.value) }
  return fail('інтервал, число або «(»')
}

function unary(): Interval {
  if (isSym('+')) {
    p++
    return unary()
  }
  if (isSym('-')) {
    p++
    const a = unary()
    const result = a.neg()
    trace.push({ kind: 'neg', a, result })
    return result
  }
  return primary()
}

function term(): Interval {
  let v = unary()
  while (isSym('*', '/')) {
    const op = (tokens[p++] as { value: BinaryOp }).value
    const pos = peek().pos
    v = apply(op, v, unary(), pos)
  }
  return v
}

function expr(): Interval {
  let v = term()
  while (isSym('+', '-')) {
    const op = (tokens[p++] as { value: BinaryOp }).value
    const pos = peek().pos
    v = apply(op, v, term(), pos)
  }
  return v
}

if (peek().kind === 'end') throw new ParseError('Вираз порожній', 0)
let value: Interval
try {
  value = expr()
} catch (e) {
  if (e instanceof ParseError) throw e
```

```
    if (e instanceof RangeError) throw new ParseError('Вираз занадто глибоко
    вкладений', 0)
    throw new ParseError((e as Error).message, peek().pos)
  }
  if (peek().kind !== 'end') fail('оператор або кінець виразу')
  return { value, trace, warnings }
}
```

Файл complex.ts

```
// Комплексний інтервал у прямокутній формі A = A1 + i·A2
import { Interval, type IntervalLike, type Warn } from './interval'

export class ComplexInterval {
  readonly re: Interval
  readonly im: Interval

  constructor(re: IntervalLike, im: IntervalLike) {
    this.re = Interval.from(re)
    this.im = Interval.from(im)
  }

  static readonly ZERO = new ComplexInterval(0, 0)

  /** A + B = (A1 + B1) + i(A2 + B2) */
  add(b: ComplexInterval): ComplexInterval {
    return new ComplexInterval(this.re.add(b.re), this.im.add(b.im))
  }

  /** A − B = (A1 − B1) + i(A2 − B2) */
  sub(b: ComplexInterval): ComplexInterval {
    return new ComplexInterval(this.re.sub(b.re), this.im.sub(b.im))
  }

  /** A · B = (A1B1 − A2B2) + i(A1B2 + A2B1) */
  mul(b: ComplexInterval): ComplexInterval {
    return new ComplexInterval(
      this.re.mul(b.re).sub(this.im.mul(b.im)),
      this.re.mul(b.im).add(this.im.mul(b.re)),
    )
  }

  /** A ÷ B = (A1B1 + A2B2)/(B1² + B2²) + i(A2B1 − A1B2)/(B1² + B2²) */
  div(b: ComplexInterval, warn?: Warn): ComplexInterval {
    const den = b.re.sqr().add(b.im.sqr())
    return new ComplexInterval(
      this.re.mul(b.re).add(this.im.mul(b.im)).div(den, warn),
      this.im.mul(b.re).sub(this.re.mul(b.im)).div(den, warn),
    )
  }

  equals(b: ComplexInterval, eps = 1e-9): boolean {
    return this.re.equals(b.re, eps) && this.im.equals(b.im, eps)
  }

  toString(digits?: number): string {
    return `${this.re.toString(digits)} + i·${this.im.toString(digits)}`
  }
}
```

Файл interval.ts

```
// Дійсний інтервал [a1; a2]
import { fmtHi, fmtLo } from './format'

export type Warn = (message: string) => void
export type IntervalLike = Interval | readonly [number, number] | number

export class Interval {
  readonly lo: number
  readonly hi: number

  constructor(lo: number, hi: number = lo) {
    if (!Number.isFinite(lo) || !Number.isFinite(hi)) throw new Error('Межі
    інтервалу мають бути скінченними числами')
    if (lo > hi) throw new Error(`Некоректний інтервал [${lo}; ${hi}]: ліва межа
    більша за праву`)
    this.lo = lo + 0 // −0 → 0
    this.hi = hi + 0
  }

  static from(x: IntervalLike): Interval {
    if (x instanceof Interval) return x
    if (typeof x === 'number') return new Interval(x, x)
    return new Interval(x[0], x[1])
  }

  /** A + B = [a1 + b1; a2 + b2] */
  add(other: IntervalLike): Interval {
    const b = Interval.from(other)
    return new Interval(this.lo + b.lo, this.hi + b.hi)
  }

  /** A − B = [a1 − b2; a2 − b1] */
  sub(other: IntervalLike): Interval {
    const b = Interval.from(other)
    return new Interval(this.lo - b.hi, this.hi - b.lo)
  }

  /** Чотири добутки кінців {a1b1, a1b2, a2b1, a2b2} */
  products(other: IntervalLike): number[] {
    const b = Interval.from(other)
    return [this.lo * b.lo, this.lo * b.hi, this.hi * b.lo, this.hi * b.hi]
  }

  /** Чотири частки кінців {a1/b1, a1/b2, a2/b1, a2/b2} */
  quotients(other: IntervalLike): number[] {
    const b = Interval.from(other)
    return [this.lo / b.lo, this.lo / b.hi, this.hi / b.lo, this.hi / b.hi]
  }

  /** A · B = [min{a1b1, a1b2, a2b1, a2b2}; max{…}] */
  mul(other: IntervalLike): Interval {
    const p = this.products(other)
    return new Interval(Math.min(...p), Math.max(...p))
  }

  /**
   * A / B = [min{a1/b1, a1/b2, a2/b1, a2/b2}; max{…}]
```

```
   * 0 усередині B: те саме правило, що в лекції (приклад 3), з попередженням
    через warn.
   */
  div(other: IntervalLike, warn?: Warn): Interval {
    const b = Interval.from(other)
    if (b.lo === 0 || b.hi === 0) throw new Error(`Ділення на ${b} неможливе: межа
    дільника дорівнює 0`)
    if (b.lo < 0 && b.hi > 0) warn?.(`дільник ${b} містить 0 — застосовано правило
    min/max чотирьох часток (лекція 1, приклад 3)`)
    const q = this.quotients(b)
    return new Interval(Math.min(...q), Math.max(...q))
  }

  /** −A = [−a2; −a1] */
  neg(): Interval {
    return new Interval(-this.hi, -this.lo)
  }

  /** {x² | x ∈ A} */
  sqr(): Interval {
    const a = this.lo * this.lo
    const b = this.hi * this.hi
    return this.contains(0) ? new Interval(0, Math.max(a, b)) : new
    Interval(Math.min(a, b), Math.max(a, b))
  }

  contains(x: number): boolean {
    return this.lo <= x && x <= this.hi
  }

  /** wid(A) = a2 − a1 */
  get width(): number {
    return this.hi - this.lo
  }

  /** mid(A) = (a1 + a2) / 2 */
  get mid(): number {
    return (this.lo + this.hi) / 2
  }

  equals(other: IntervalLike, eps = 1e-9): boolean {
    const b = Interval.from(other)
    return Math.abs(this.lo - b.lo) <= eps && Math.abs(this.hi - b.hi) <= eps
  }

  toString(digits?: number): string {
    return `[${fmtLo(this.lo, digits)}; ${fmtHi(this.hi, digits)}]`
  }
}
```

Файл format.ts

```
// Межі округлюються назовні (ліва вниз, права вгору), щоб показаний інтервал
    містив точний

export const DIGITS = 4

const noMinusZero = (x: number): number => (Object.is(x, -0) ? 0 : x)
```

```
function roundDir(x: number, digits: number, dir: 'down' | 'up'): number {
  const k = 10 ** digits
  const scaled = x * k
  if (!Number.isFinite(scaled)) return x
  const nearest = Math.round(scaled)
  // Не розширюємо інтервал через похибку float (0.1 + 0.2)
  if (Math.abs(scaled - nearest) < 1e-9 * Math.max(1, Math.abs(scaled))) return
    noMinusZero(nearest / k)
  return noMinusZero((dir === 'down' ? Math.floor(scaled) : Math.ceil(scaled)) /
    k)
}

export const fmtLo = (x: number, digits = DIGITS): string => String(roundDir(x,
    digits, 'down'))
export const fmtHi = (x: number, digits = DIGITS): string => String(roundDir(x,
    digits, 'up'))
export const fmtNum = (x: number, digits = DIGITS): string =>
    String(noMinusZero(Number(x.toFixed(digits))))
```